

Computer Graphics #2 “geometry processing”

Manon Vu Huu

2025-2026

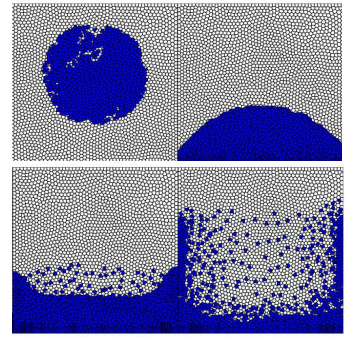
DURATION:

3 labs, only during the 6 TD hours

GLOBAL GOAL:

Build a free-surface 2D fluid solver using:

- Voronoi diagrams
- Sutherland-Hodgman polygon clipping
- Power diagrams
- Semi-discrete optimal transport with L-BFGS
- Partial optimal transport for air + fluid
- A final animation of moving fluid particles



Eventually manage to get something like this →

Professor's image

STEP 0: create a github repo https://github.com/j3am6s/computer_graphics_manon

LAB 1: (struggling to understand the geometry and feeling confused)

LOCAL GOAL:

“implement the Voronoi Parallel Linear Enumeration algorithm in 2D, at least the naive $O(N^2)$ version, using Sutherland-Hodgman polygon clipping”

STEP 0: introduction

A Voronoi diagram separates the unit square into cells, each corresponding to a point. The cell of a site contains all the positions that are closer to this site than to the others.

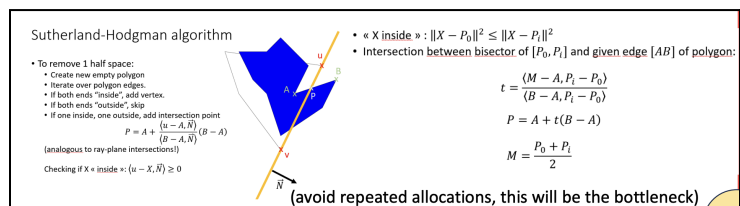
So for every point, we start with the whole square, then cut away the parts that belong to the other points. If the clipping works, then the Voronoi diagram is basically obtained by repeating the clipping many times.

STEP 1: Sutherland-Hodgman for clipping by a bisector

The first step is to implement Sutherland-Hodgman polygon clipping.

The goal is to remove one half-space from a polygon.

This was the first complicated part because if the inside/outside test is reversed the program still draws something, but it's the wrong half of the polygon. Also, if there are repeated allocations or unnecessary copies, this function becomes slow.



STEP 2: Voronoi diagram

Once clipping by one bisector works, the Voronoi diagram becomes a repetition of the same operation. For each site: we start with the full unit square, compare the current site with every other site, clip the current polygon by the bisector between the two sites, and continue until all other sites have been used. At the end, the remaining polygon is the Voronoi cell of the current site.

We implement the naive $O(N^2)$ version, so every site is compared with every other site. The optional optimized version would use Nanoflann and only clip with the k-nearest neighbors, but we stayed with the simpler version.

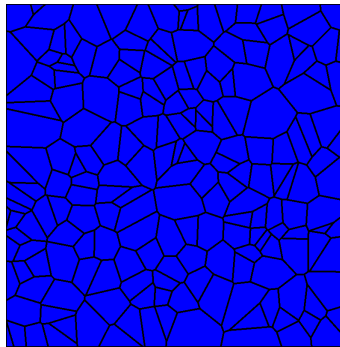
This is also why the algorithm is called Parallel Linear Enumeration: every cell can be computed independently, so the loop over all sites can be parallelized.

STEP 3: test on random points

To test the implementation, we generate random points inside the unit square and computed their Voronoi cells. Random points quickly show if something is wrong:

- if cells go outside the square: the initial polygon or clipping is wrong
- if cells overlap too much: the bisector test is wrong
- if holes appear: the intersection cases are wrong
- if some cells disappear: the inside/outside sign is probably wrong

We are now able to generate a full Voronoi diagram in 2D using random sites.



LAB 7: (feeling more confident)

LOCAL GOAL:

“implement semi-discrete optimal transport in 2D using L-BFGS between weighted Diracs and a uniform density”

STEP 1: adapt Sutherland-Hodgman for Power diagrams

First we modify the clipping from lab 6.

This was nice because the structure of Sutherland-Hodgman does not change. Only the definition of the bisector and the inside test do.

- New « Middle » point $M' = M + \frac{w_0 - w_i}{2\|P_0 - P_i\|^2} (P_i - P_0)$
- Intersection point P, still: $t = \frac{\langle M' - A, P_i - P_0 \rangle}{\langle B - A, P_i - P_0 \rangle}$ and $P = A + t(B - A)$
- « X inside » : $\|X - P_0\|^2 - w_0 \leq \|X - P_i\|^2 - w_i$

STEP 2: semi-discrete optimal transport objective function and gradient

We then need to optimize the weights. The objective function from the lecture notes is maximized, and contains 2 main parts: an integral over each power cell, and a term with the prescribed masses λ_i and the weights. The important thing is that the area of each power cell appears in the gradient.

- Maximize functional: $g(W) = \sum_i \int_{\text{Pow}_W(y_i)} (\ x - y_i\ ^2 - w_i) f(x) dx + \sum_i \lambda_i w_i$ - Gradient : $\frac{\partial g}{\partial w(y_i)}(W) = \lambda_i - \int_{\text{Pow}_W(y_i)} f(x) dx$	Area polygon in 2d (seen before): $A = \frac{1}{2} \sum_{i=0}^{N-1} (x_i y_{i+1} - x_{i+1} y_i)$ $\int_T \ x - P_i\ ^2 dx = \frac{ T }{6} \sum_{1 \leq k \leq l \leq 3} \langle c_k - P_i, c_l - P_i \rangle$
---	---

(The gradient is as such because the density is uniform in this project.)

STEP 3: optimize in the L-BFGS solver

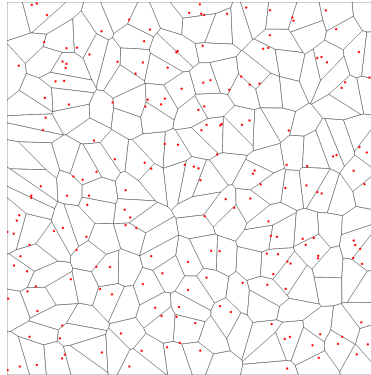
After writing the objective and gradient, we plug everything into the L-BFGS solver. For the basic optimal transport test, we use equal masses: **lambda_i = 1 / n**

This means every cell should converge toward the same area.

The solver calls evaluate: at every call we update the weights, recompute the power diagram, and the gradient tells the optimizer how far each cell is from its target area.

If a cell is too big (area > lambda_i) the gradient pushes the weight in the direction that makes the cell smaller. If a cell is too small, vice versa.

To check that it works, we test different weight configurations. When the weights are all equal, the diagram looks like a normal Voronoi diagram. When the weights change depending on the distance to the center, the cells also change size: power diagram & optimization are being used.



LAB 8: (most fun part)

LOCAL GOAL:

“implement the semi-discrete optimal transport fluid simulator with free surfaces”

STEP 1: adapt semi-discrete OT code for partial transport

For a free-surface fluid we do not want the optimal transport to fill the whole unit square. The fluid should only occupy a part, and the rest should be air. So the first thing to change was the total mass. Instead of having a total mass of 1, we choose a desired fluid volume. In the code, we use **fluid_volume = 0.6**. For air, we add **w_air**. So the weights are not only the weights of the fluid particles anymore. There is also an additional air weight at the end.

Then, each cell is intersected with a disk. The radius depends on the difference between the particle weight and the air weight. In practice, we discretize the disk with 100 vertices, then we reuse the Sutherland-Hodgman idea to clip the cell with the disk edges. So this step is connected to lab 1: if clipping works for half-planes, we can adapt it for the disk boundary.

The optimal transport energy also has to be adapted. The target area is no longer $1 / N$ but $\text{desired_vol_fluid} / N_{\text{fluid}}$. So the gradient for each fluid particle compares the cell area with the target one. We also add air gradient, which compares desired air volume with estimated air volume.

$$g(W) = \sum_{i=1}^{N_{\text{fluid}}} \int_{\text{Pow}_i(y_i)}^{\square} (\|x - y_i\|^2 - w_i) dx + \sum_{i=1}^{N_{\text{fluid}}} \frac{\text{desired vol}_{\text{fluid}}}{N_{\text{fluid}}} w_i + w_{\text{air}} (\text{desired vol}_{\text{air}} - \text{estimated vol}_{\text{air}})$$

$$\frac{\partial g}{\partial w_i}(W) = \frac{\text{desired vol}_{\text{fluid}}}{N_{\text{fluid}}} - \text{Area}(\text{Pow}_i) \quad \text{for } 1 \leq i \leq N_{\text{fluid}}$$

$$\frac{\partial g}{\partial w(\text{air})}(W) = \text{desired vol}_{\text{air}} - \text{estimated vol}_{\text{air}}$$

STEP 2: run particle simulation

At each time step, we first compute the optimal transport from the particle positions to the uniform density. This gives the current Laguerre cells of the particles. Then for each particle we compute a spring force that pulls the particle toward the centroid of its cell. So the force has two parts: the spring force (moves the particle toward the centroid of its Laguerre cell), and the gravity (pulls everything down). Then we update the velocity, and after that the position.

In the final code we use:

$\epsilon = 0.004$

$m_i = 200$

$dt = 0.006$

- $F_{\text{spring}}^i = \frac{1}{\epsilon^2} (\text{Centroid}(\text{Laguerre}_i) - X_i^t)$
- $F^i = F_{\text{spring}}^i + m_i \vec{g}$
- $v_i^{t+1} = v_i^t + \frac{dt}{m_i} F^t$
- $X_i^{t+1} = X_i^t + dt v_i^{t+1}$
- Bounce particles back into the domain

After moving the particles, we check if they left the unit square. If yes, we put it back inside and reverse its velocity with damping. This makes it bounce back instead of disappearing.

This step finally makes the result feel like a fluid simulation, because the geometry is recomputed at every frame and the particles move according to their cells.

